

SOFTWARE REQUIREMENTS SPECIFICATION

Project: Real-Time Safety Equipment & Anomaly Detection System **Version:** 1.1 | **Date:** April 2026 | **Course:** Advanced Programming

1. INTRODUCTION

1.1 Purpose

This document defines the software requirements for the Real-Time Safety Equipment & Anomaly Detection System. The system aims to enhance workplace safety by monitoring multiple video streams in real-time to detect missing Personal Protective Equipment (PPE) and physical anomalies such as falls or fainting.

1.2 Scope

The software is a desktop-based application utilizing YOLO-based computer vision and deep learning, built with PyQt6. It manages multiple asynchronous video feeds, autonomously flags violations, triggers visual/audio alarms, and logs events for future auditing.

1.3 Definitions and Abbreviations

Term	Definition
PPE	Personal Protective Equipment (hardhat, vest, goggles, etc.)
YOLO	You Only Look Once — real-time object detection architecture
RTSP	Real Time Streaming Protocol — IP camera stream protocol
MVC	Model-View-Controller design pattern
FPS	Frames Per Second
mAP	Mean Average Precision — model accuracy metric
QThread	Qt-based thread class used for non-blocking operations

2. OVERALL DESCRIPTION

2.1 User Characteristics

Primary users are security personnel, site managers, and occupational safety officers. The interface is designed to be intuitive, requiring no technical background to monitor the dashboard or read the anomaly logs.

2.2 Operating Environment

- **OS:** Windows 10/11 or Linux (Ubuntu 20.04+)
- **Hardware:** Multi-core CPU; NVIDIA CUDA-capable GPU strongly recommended for real-time inference
- **Camera Support:** IP Cameras (RTSP streams), USB Webcams, MP4/AVI video files

2.3 General Constraints

- The system operates fully offline; no internet connection required
- Inference can run on CPU but performance will be significantly reduced without a GPU
- Training dataset must be sourced and annotated externally

3. FUNCTIONAL REQUIREMENTS (FR)

3.1 Video & Stream Management

FR-1.1: The system must allow users to add, remove, and manage multiple video stream sources concurrently.

FR-1.2: The GUI must automatically arrange active video streams in a responsive grid layout.

FR-1.3: The system must support RTSP (IP camera), USB webcam, and local video file (MP4, AVI) sources.

3.2 AI & Computer Vision Modules

FR-2.1 — PPE Detection: The system must utilize a custom-trained YOLO model to detect humans and verify the presence of safety equipment (hardhats, safety vests) within bounding boxes.

FR-2.2 — Anomaly / Fall Detection: The system must integrate a YOLOv8-pose skeleton analysis module to identify irregular human postures indicative of fainting or falling.

FR-2.3 — Threshold Configuration: The system must allow users to adjust confidence thresholds (e.g., 0.50 to 0.85) for all detection models via a settings menu.

FR-2.4 — Multi-Class Extensibility: The system must support new equipment classes (goggles, gloves, etc.) without modifying the core video processing logic.

3.3 Alerting & Spotlight System

FR-3.1: Upon detecting a rule violation (e.g., "Person without Hardhat" or "Fall Detected"), the system must change the state of the specific video cell.

FR-3.2: The GUI must apply a prominent visual indicator — a flashing red "spotlight" border — to the video feed where the anomaly is occurring.

FR-3.3: The system must trigger an audible alarm when a high-priority anomaly is detected.

FR-3.4: The alarm logic must include a throttle mechanism to prevent repeated alerts for the same camera within a configurable cooldown period (default: 5 seconds).

3.4 Event Logging & Reporting

FR-4.1: All detected anomalies must be recorded in an on-screen scrolling log panel (Event Bus UI).

FR-4.2: The system must save log data — Timestamp, Camera ID, Anomaly Type, Confidence Score — to a local CSV or JSON file for historical tracking.

FR-4.3: The system must automatically capture and save a screenshot of the frame in which the anomaly was detected.

4. NON-FUNCTIONAL REQUIREMENTS (NFR)

4.1 Performance & Concurrency

NFR-1.1 — Multi-threading: The system architecture must separate the GUI main loop, video capture loops, and AI inference tasks into distinct threads using QThread or Python threading.

NFR-1.2 — Latency & FPS: The processing pipeline must be optimized to prevent GUI freezing, maintaining a minimum display frame rate of 15–25 FPS per camera (hardware dependent).

4.2 Architecture & Maintainability

NFR-2.1 — Design Patterns: The source code must implement MVC architecture, Strategy Pattern (for swappable AI models), and Observer Pattern (for decoupled event handling).

NFR-2.2 — Extensibility: The codebase must be modular, allowing developers to integrate new detection classes without rewriting core video processing logic.

4.3 Reliability & Error Handling

NFR-3.1 — Stream Recovery: If a video stream disconnects, the system must handle the exception gracefully, display a "Connection Lost" placeholder, and attempt to reconnect automatically without crashing.

NFR-3.2 — Resource Management: The system must properly manage memory and prevent memory leaks when running video loops continuously over extended periods.

NFR-3.3 — Graceful Shutdown: The system must safely terminate all threads and release camera resources when the application is closed.

5. PROJECT STRUCTURE

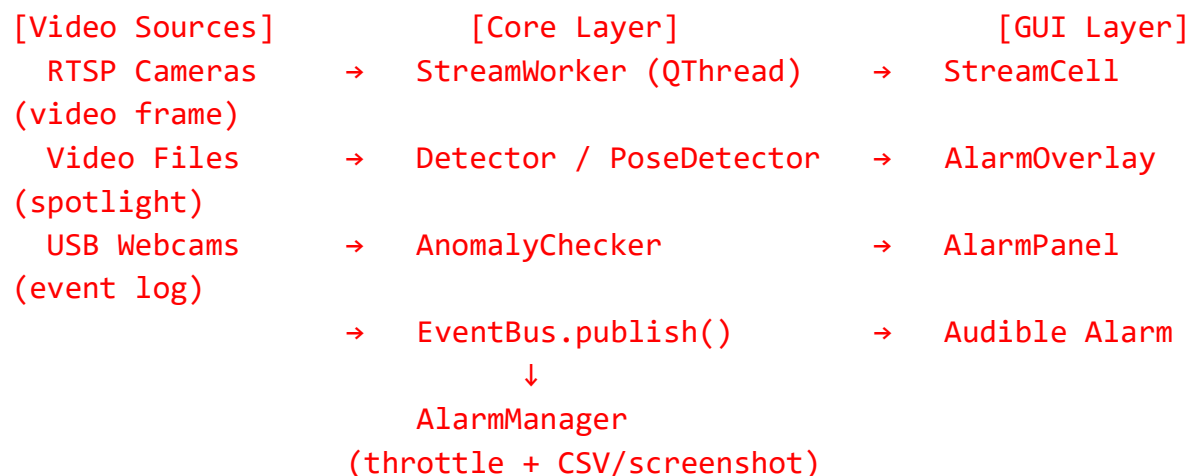
```
safety_detection/  
|  
├─ main.py  
├─ config/  
|   └─ settings.py  
|   └─ equipment_classes.yaml
```

```
├── core/
│   ├── detector.py           # YOLO PPE wrapper (Strategy Pattern)
│   ├── pose_detector.py      # YOLOv8-pose fall detection (NEW)
│   ├── stream_worker.py      # Per-camera QThread
│   ├── alarm_manager.py      # Alarm throttle + audio trigger
│   └── event_bus.py          # Pub/sub Observer system
├── gui/
│   ├── main_window.py        # Root window + grid layout
│   ├── stream_cell.py        # Single video widget
│   ├── alarm_overlay.py      # Flashing red spotlight border
│   ├── alarm_panel.py        # Scrolling event log panel
│   └── settings_dialog.py    # Threshold & source config dialog
├── utils/
│   ├── draw.py               # Bounding box helpers
│   ├── logger.py             # CSV/JSON log writer (FR-4.2)
│   ├── screenshot.py         # Anomaly frame capture (FR-4.3) NEW
│   └── video_source.py       # RTSP/webcam/file abstraction
├── models/
│   ├── best.pt               # Custom PPE weights
│   └── yolov8n-pose.pt       # Pose model for fall detection
├── training/
│   ├── train.py
│   └── validate.py
├── data/
│   ├── dataset/
│   └── data.yaml
├── tests/
│   ├── test_detector.py
│   ├── test_alarm_manager.py
│   └── test_stream_worker.py
├── requirements.txt
└── README.md
```

6. DESIGN PATTERNS

Pattern	Where Applied	Benefit
MVC	core (Model), gui (View), stream_worker (Controller)	Separates UI from logic; enables isolated testing
Strategy	detector.py / pose_detector.py behind IDetector interface	Swap YOLO models without changing pipeline
Observer	event_bus.py publishes events; GUI widgets subscribe	Fully decoupled alarm-to-UI communication
Factory	StreamWorkerFactory creates thread instances per source	Consistent lifecycle management
Thread-per-stream	Each camera runs in its own QThread with frame queue	Non-blocking GUI; parallel multi-camera inference

7. DATA FLOW



8. TECHNOLOGY STACK

Component	Technology
-----------	------------

Language	Python 3.9+
Deep Learning	PyTorch + Ultralytics YOLOv8 / YOLO11
Computer Vision	OpenCV (cv2)
GUI Framework	PyQt6 or PySide6
PPE Detection	Custom-trained YOLOv8 (best.pt)
Fall Detection	YOLOv8-pose (skeleton keypoint analysis)
Config Format	YAML (settings), CSV/JSON (event logs)
Threading	QThread + Python threading
Audio Alerts	pygame.mixer or playsound
Screenshot	cv2.imwrite() on anomaly frame

9. MODULES RESPONSIBILITY MATRIX

Module	Responsibility	Layer
main.py	Bootstrap Qt app and main window	Entry
config/settings.py	Global runtime config: thresholds, paths	Config
core/detector.py	YOLO inference wrapper — Strategy pattern	Model
core/pose_detector.py	YOLOv8-pose fall/faint detection	Model
core/stream_worker.py	Capture, inference, publish per camera	Model
core/alarm_manager.py	Throttling, audio trigger, event recording	Model
core/event_bus.py	Central pub/sub event system	Model
gui/main_window.py	Root window, responsive grid layout	View
gui/stream_cell.py	Renders annotated video frames	View
gui/alarm_overlay.py	Flashing red spotlight border widget	View
gui/alarm_panel.py	Scrolling on-screen event log	View
gui/settings_dialog.py	Confidence thresholds, source config	View
utils/logger.py	CSV/JSON structured event log writer	Util
utils/screenshot.py	Anomaly frame capture and save	Util
utils/video_source.py	RTSP/webcam/file source with reconnect	Util

10. REQUIREMENTS TRACEABILITY MATRIX

Requirement	Module(s)	Status
FR-1.1 Stream management	main_window.py, stream_worker.py	Planned
FR-1.2 Grid layout	main_window.py	Planned

FR-2.1 PPE Detection	core/detector.py	Planned
FR-2.2 Fall Detection	core/pose_detector.py	Planned
FR-2.3 Threshold config	settings_dialog.py, settings.py	Planned
FR-3.1 Cell state change	stream_cell.py	Planned
FR-3.2 Spotlight border	alarm_overlay.py	Planned
FR-3.3 Audible alarm	alarm_manager.py	Planned
FR-4.1 On-screen log	alarm_panel.py	Planned
FR-4.2 CSV/JSON log	utils/logger.py	Planned
FR-4.3 Screenshot capture	utils/screenshot.py	Planned
NFR-1.1 Multi-threading	stream_worker.py (QThread)	Planned
NFR-3.1 Stream recovery	utils/video_source.py	Planned